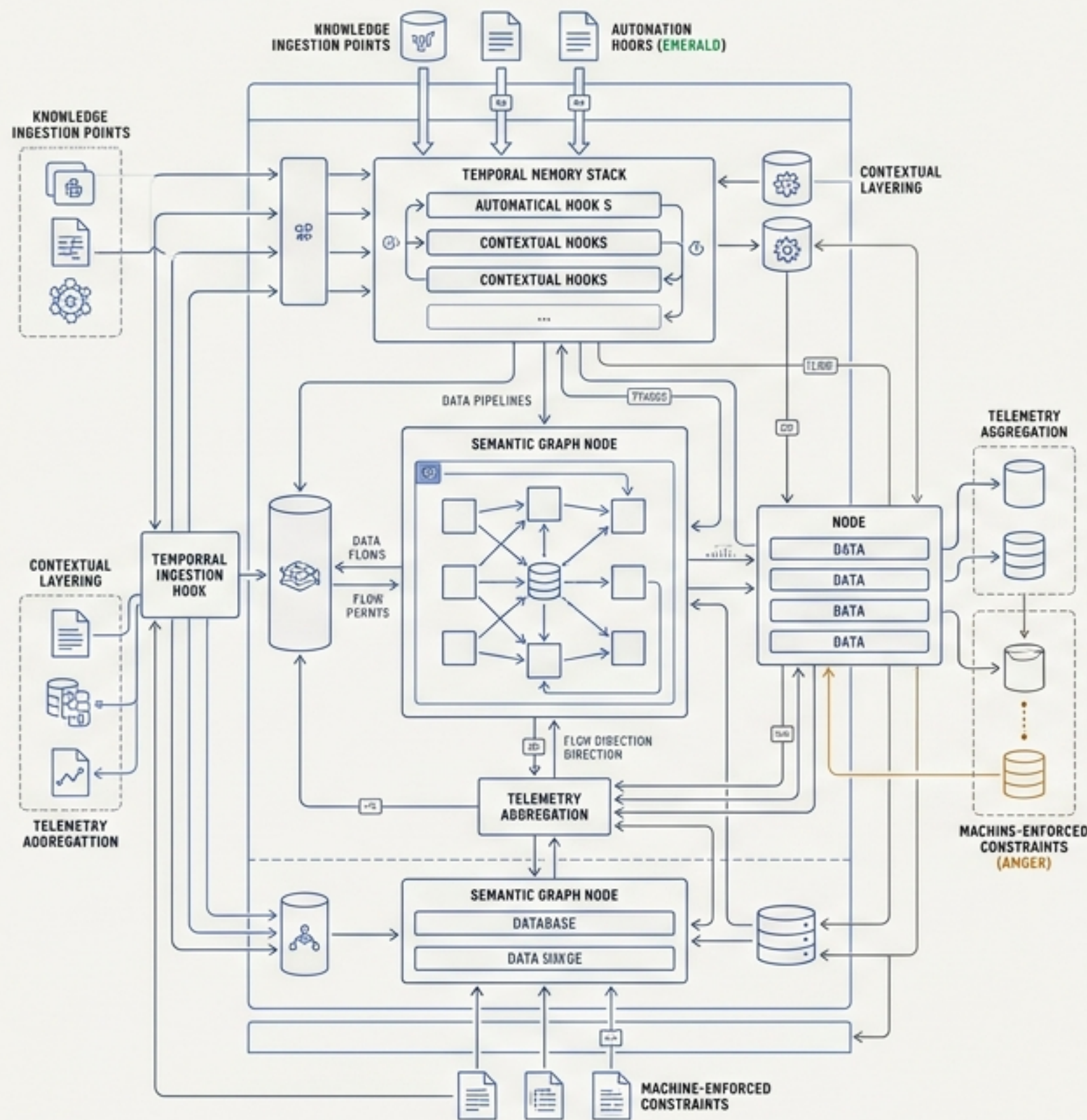


Building an Accumulative “Second Brain” on Claude Code

Architecture, Telemetry, and Evolution of a Machine-Enforced Knowledge Graph



```
> SNAPSHOT:      17 June 2026
> DEV_CYCLE:     2.2 Months (67 Days)
$ STATUS:        Executing across 14 applied projects
```


The Core Problem: Overcoming LLM Amnesia

Standard AI Chats



- **Amnesic:** Starts every conversation from zero context.



- **Repetitive:** Makes the exact same mistakes repeatedly.



- **Hallucinatory:** Fabricates facts when knowledge is missing.



- **Manual:** Requires constant human intervention and prompting.

The Accumulative Layer



- **Long-Term Memory:** Every session automatically adds verifiable knowledge.



- **Self-Correcting:** Every mistake is converted into a permanent, enforced lesson.



- **Verified:** Grounded in a machine-generated knowledge graph and executable code.



- **Autonomous:** Nightly launched timers tidy up, index, and repair without humans.

System Scale & Telemetry

Logic Engine

40

Commands (skills)

357

Interceptor hooks
(274 Python, 83 shell)

76

Background timers (launchd plists)

Knowledge Base

140

Lessons (symptom/cause/fix)

437

Knowledge atoms (K-notes)

1434

Nodes

12073

Edges in
Link Graph

Operational Footprint

74,800

Lines of code (Python + shell)

2,242

Total memory files
(264k lines, 1.95M words)

69

External repositories in registry

Architecture Overview: The 4-Loop Engine

Memory

What the system knows across 6 structured tiers.

Observation

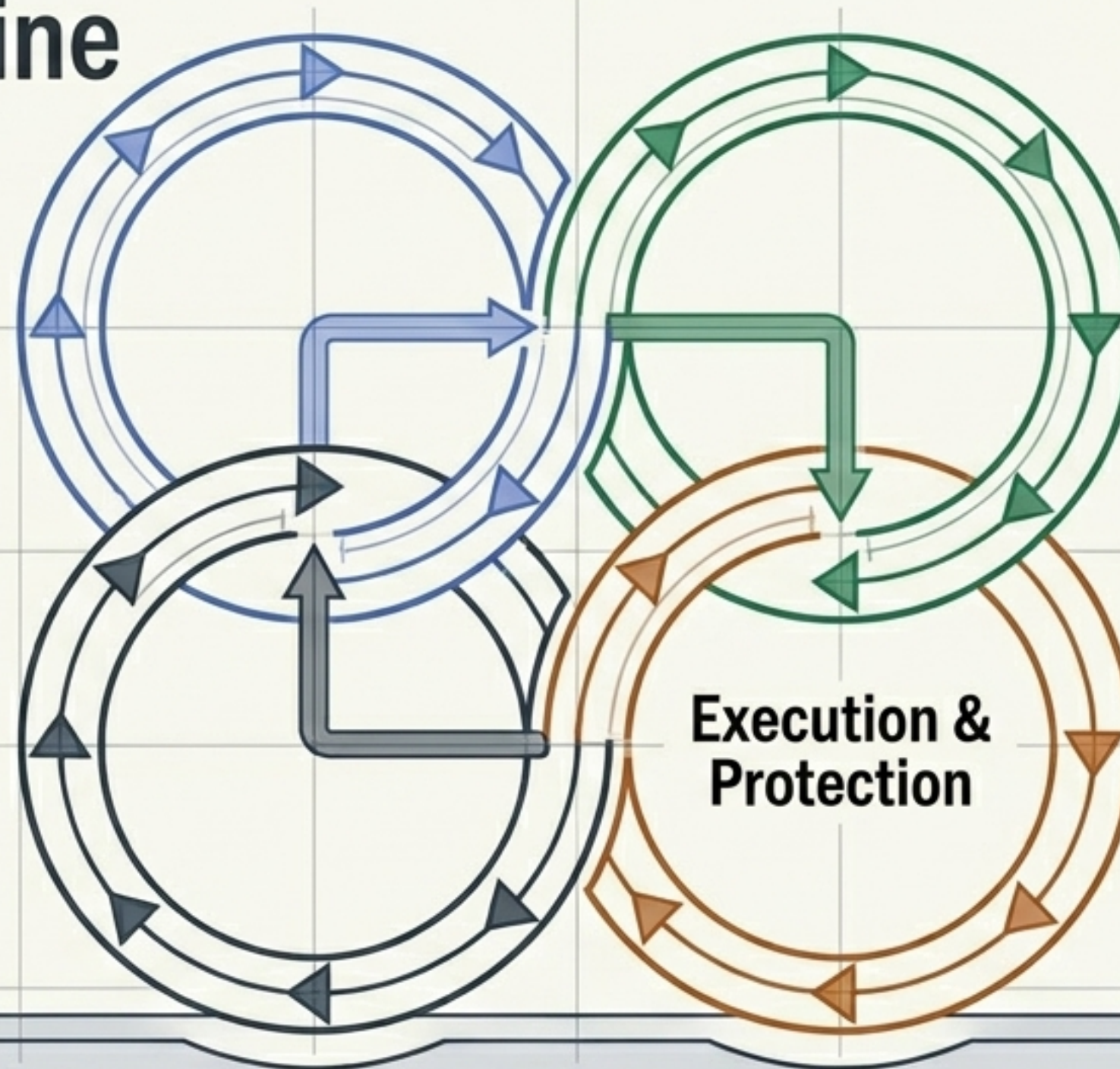
Timers, audits, and dashboards watching to ensure the first three loops stay alive.

Learning

How knowledge is born, checked, decays, and is revived across 8 phases.

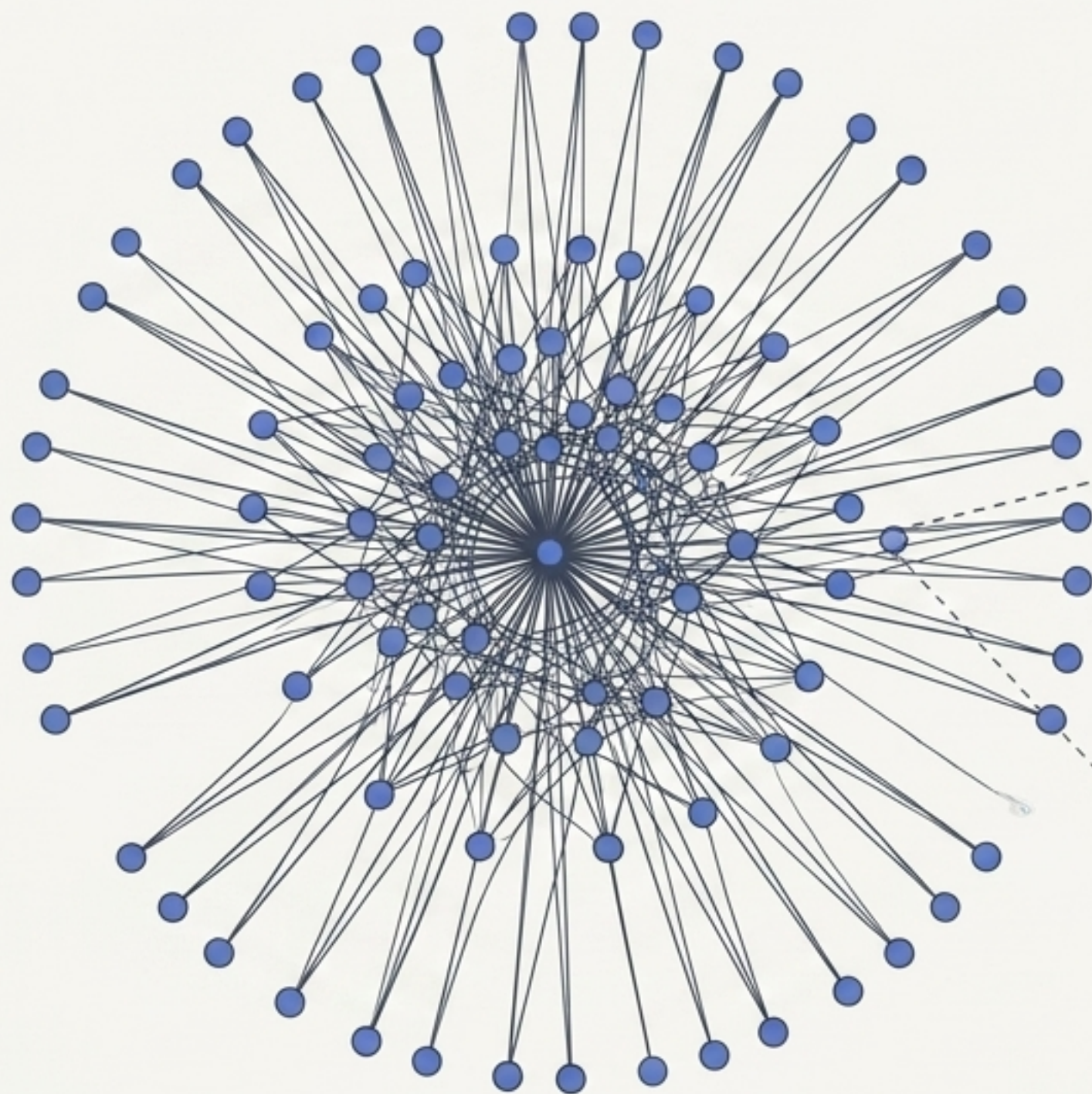
Execution & Protection

Rules and hooks keeping keeping work on protocol and blocking mistakes in real-time.



Cross-cutting Principles: Executable code grounded in knowledge theory and engineering practice.

Foundational Principles: Zettelkasten & The Knowledge Graph



Zettelkasten Mechanics

- **One file = one thought.** Folders are replaced by 27 machine-generated Maps of Content (MOC).
- A **linter** strictly catches short notes and duplicates.
- **2,789 wiki-links** across **437** notes (Average **6.38** per note; zero orphaned notes).

K-note

Title

Book teasing for Xor's and Gänote

Content Preview

Lozem ipsum dolor sit amet, content
parsed by machine--silt. Bazd
content notes, and can't see it to
astinsers shost and duplicates.

Unique ID

ID_6581278985356332453

Tags/Links

cobalt deep-recup cennett
buznt ochokes -it note

Knowledge Graph

- Directed graph architecture: 1,434 nodes and 12,073 edges.
- Follows a power-law distribution. (Top hub: a meta-security rule with 413 citations).
- Code-dependency graph built via syntax parsing, equipped with decay protection (triggers rollback if new graph is <10% volume).

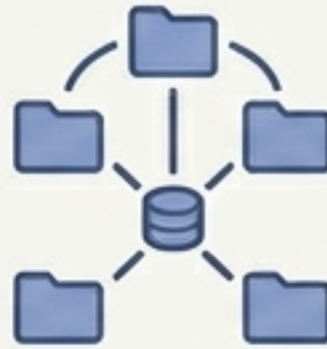
Scaling Principles: Evolution, Portability, and Cross-Project Protection

Accumulative Self-Evolution



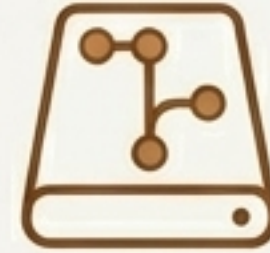
- Nightly fitness loops test code changes (up to 50 lines).
- Measures search precision, canary checks, and drift.
- Automatically rolls back via git on regression.

Cross-Project Memory



- 140 lessons and 73 rules act as a single, unified base.
- 1 lesson protects 14 projects.
- Three-tier lesson search (local, related, semantic pick).

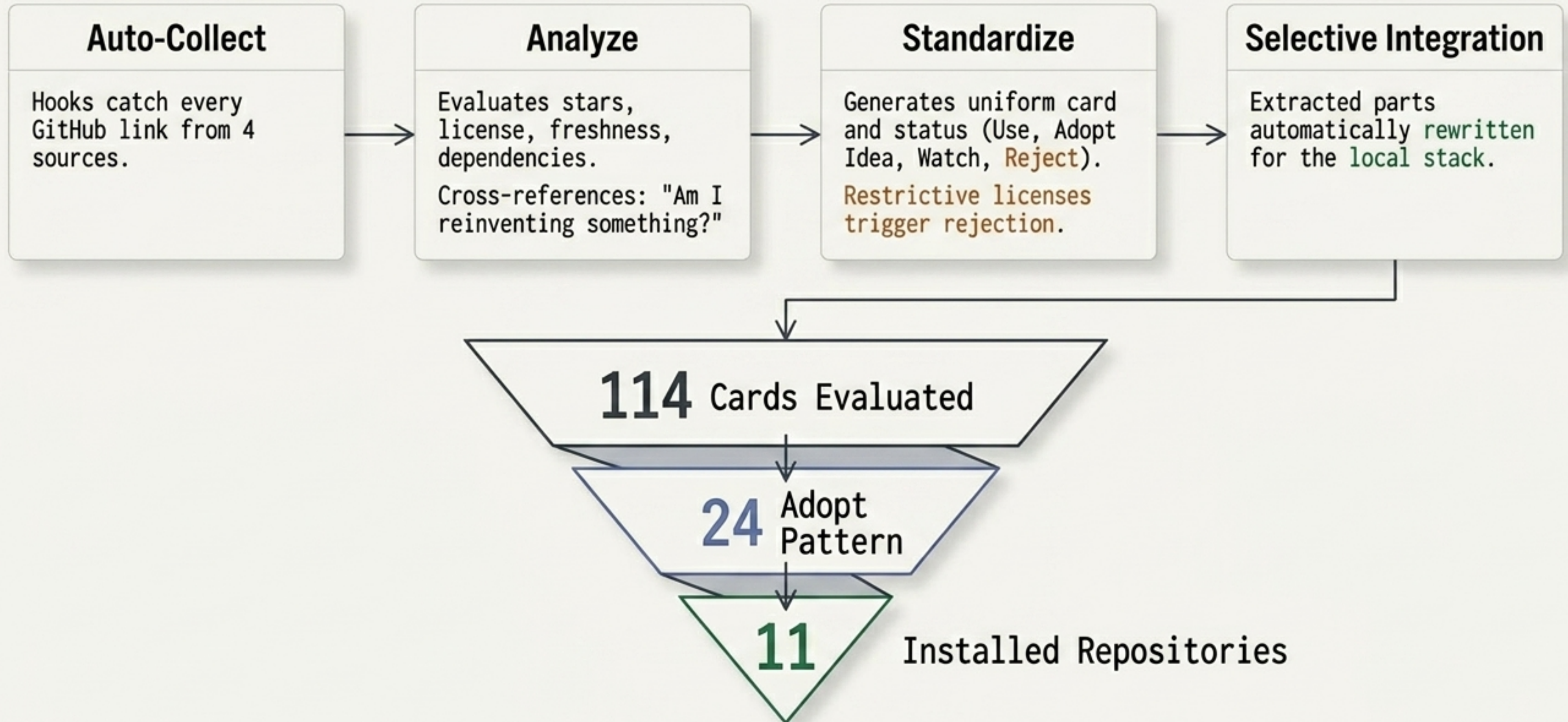
Portable Storage



- "Memory is code." Backed by git repositories.
- One-way sync after every edit; auto-commit on timer.
- Absolute paths strictly blocked to ensure machine portability.

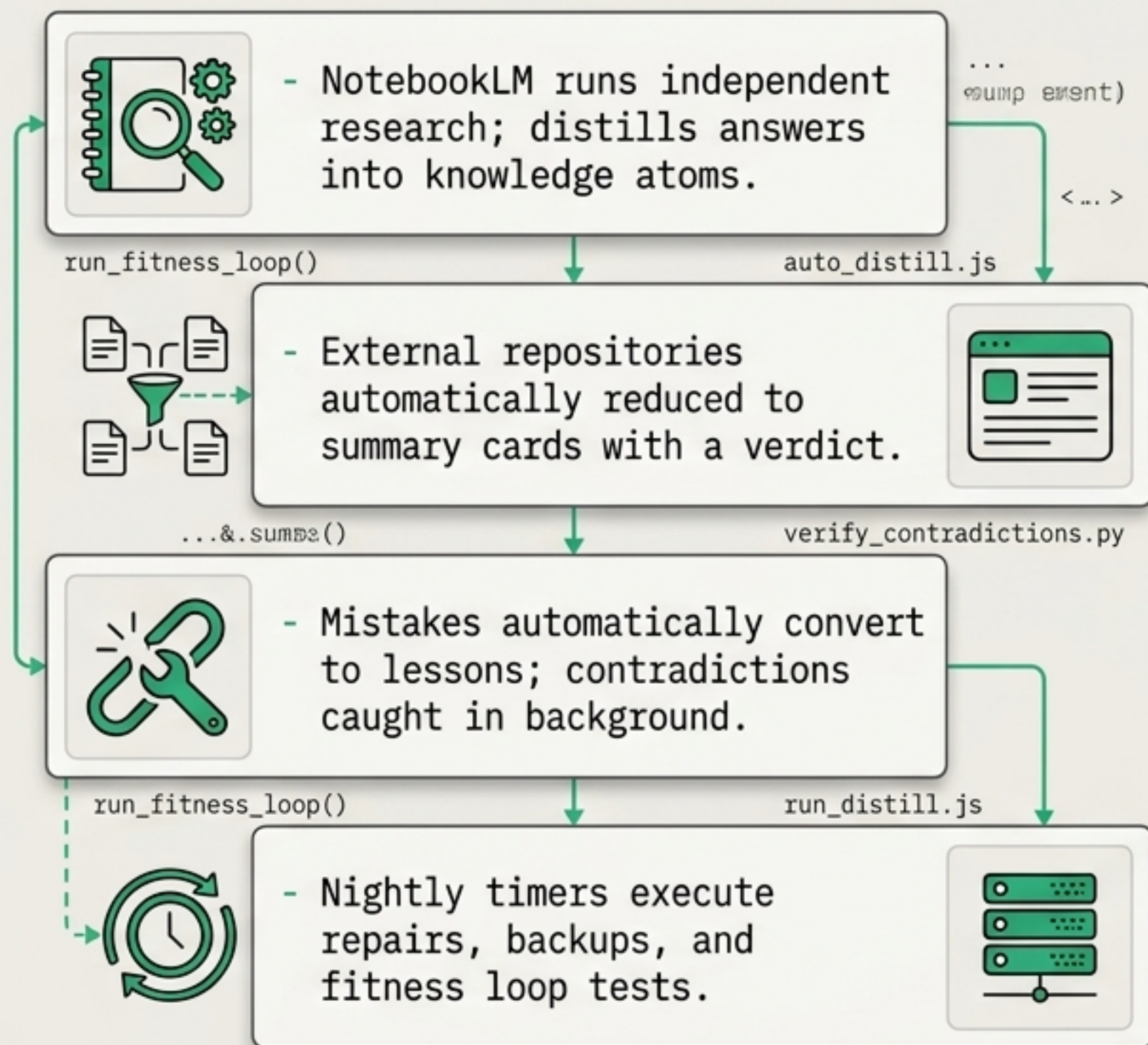
Verifiable Code Foundation

Ingestion Engine: Processing External Repositories

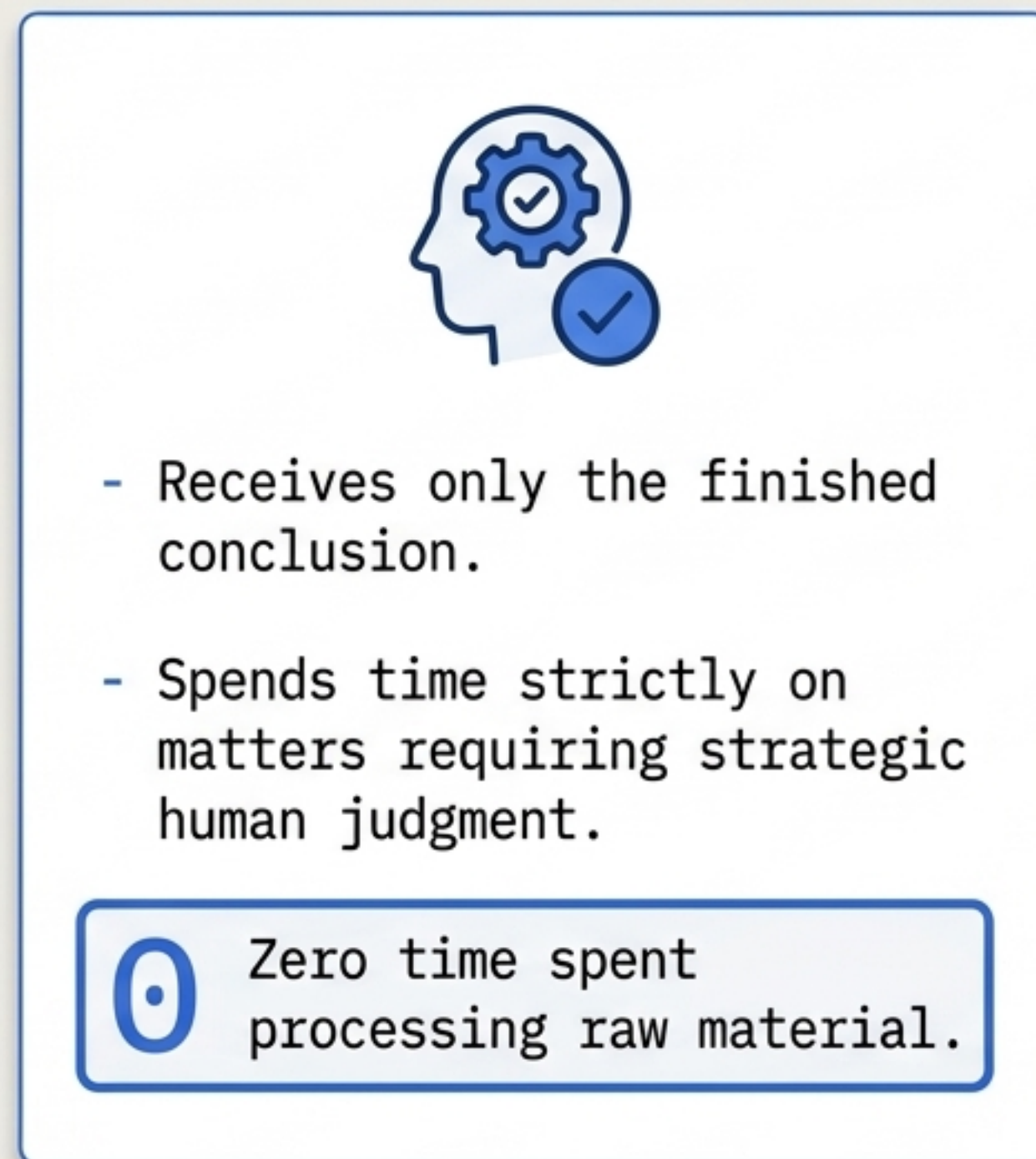


Self-Development: Doing Research Without Human Time

Machine Domain (Raw Material Processing)

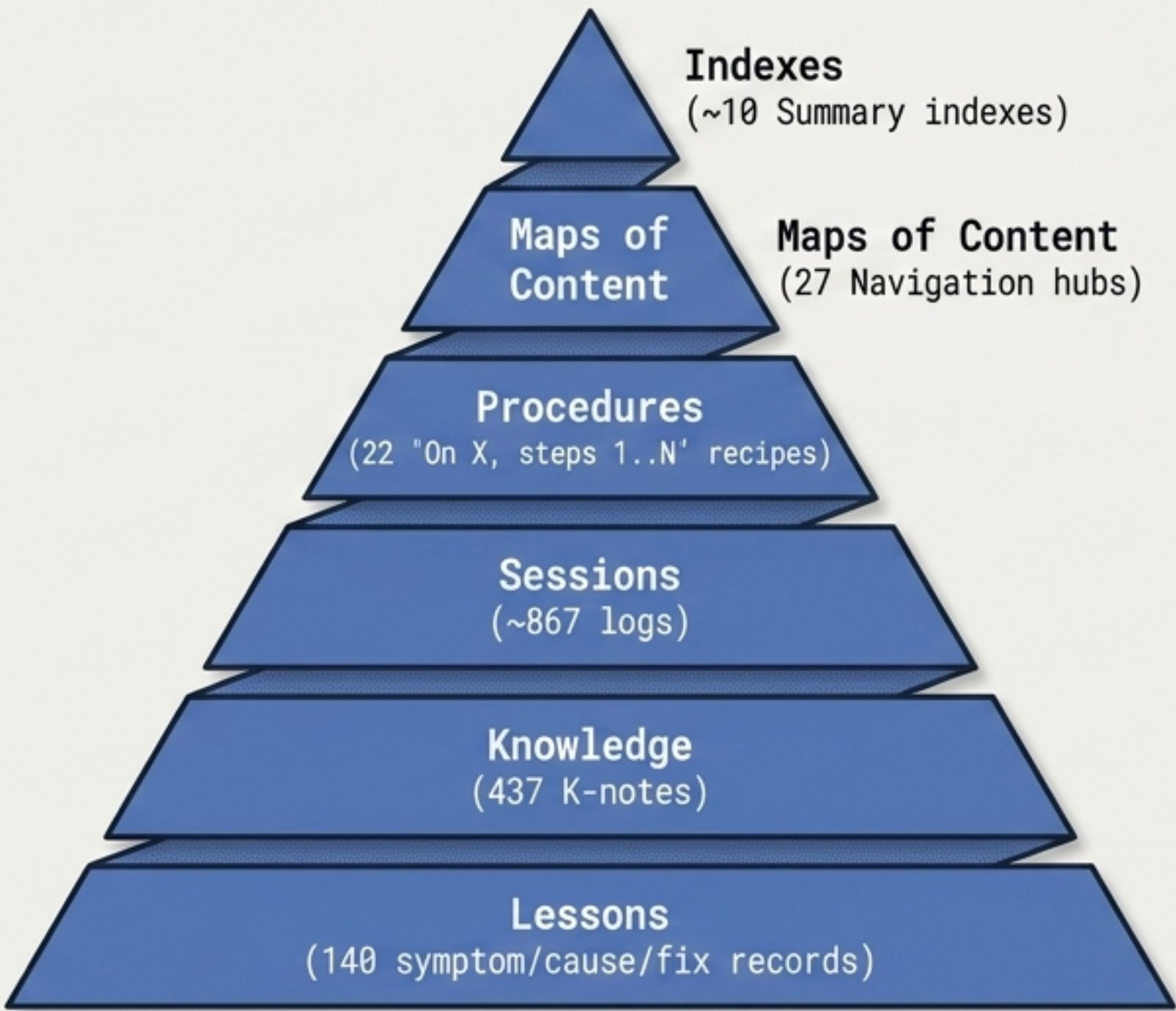


Human Domain (Strategic Judgment)

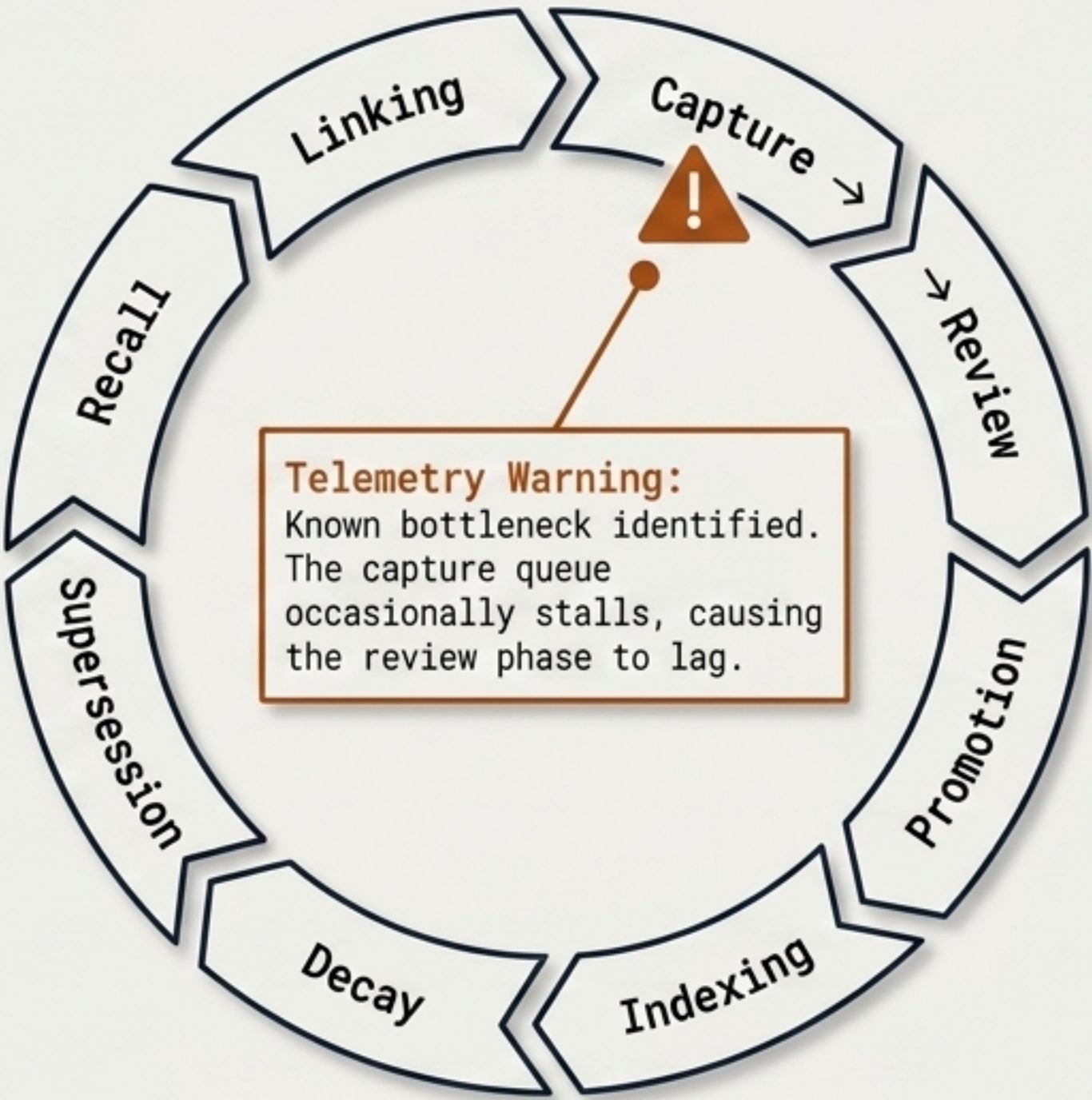


Mechanics in Practice: Memory Tiers & Learning Phases

The Memory Pyramid (6 Tiers)



The Learning Wheel (8 Phases)

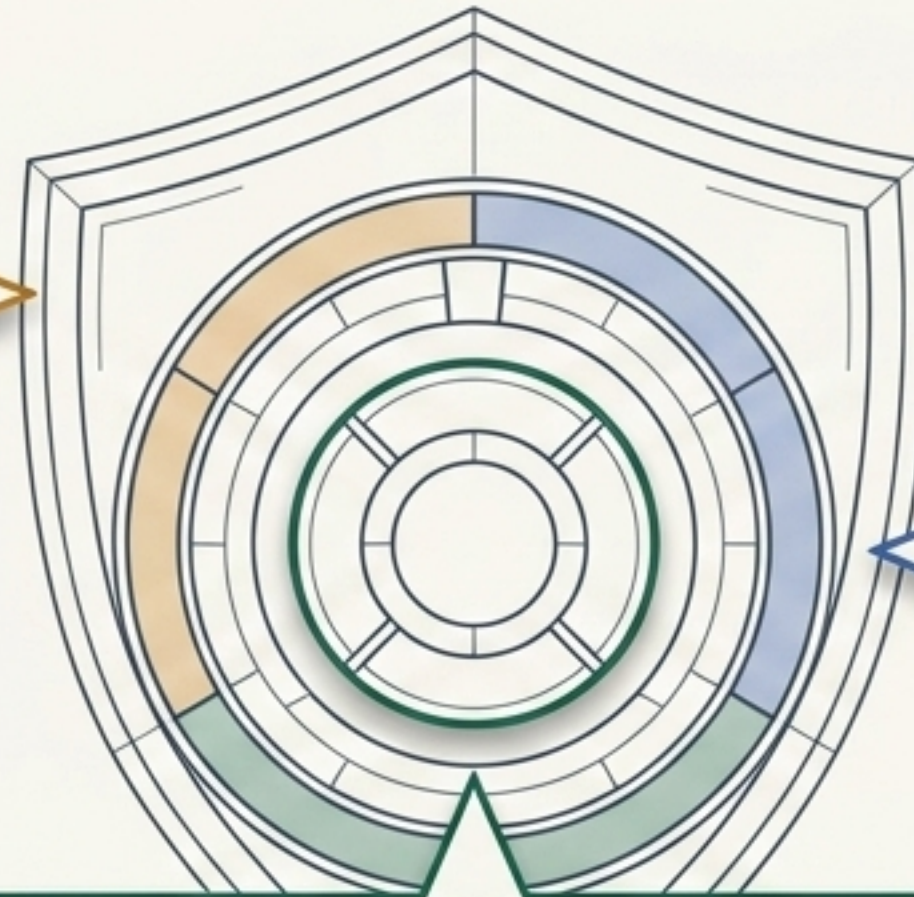


Protection Loop: Enforcing Discipline via Code

Core Philosophy: A rule you “have to remember” gets forgotten. Critical rules are converted into 357 blocking hooks.

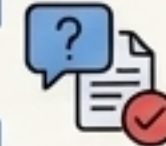
Layer 1: Before Command

Blocks destructive operations (rm -rf, force-pushes, writing to private keys).

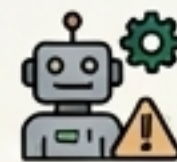


Layer 2: Before Answer / Edit

Blocks the words “done, fixed, correct” unless accompanied by verifiable command output.
Blocks unread cross-model reviews and hardcoded absolute paths.



Layer 3: Before Agent / Turn



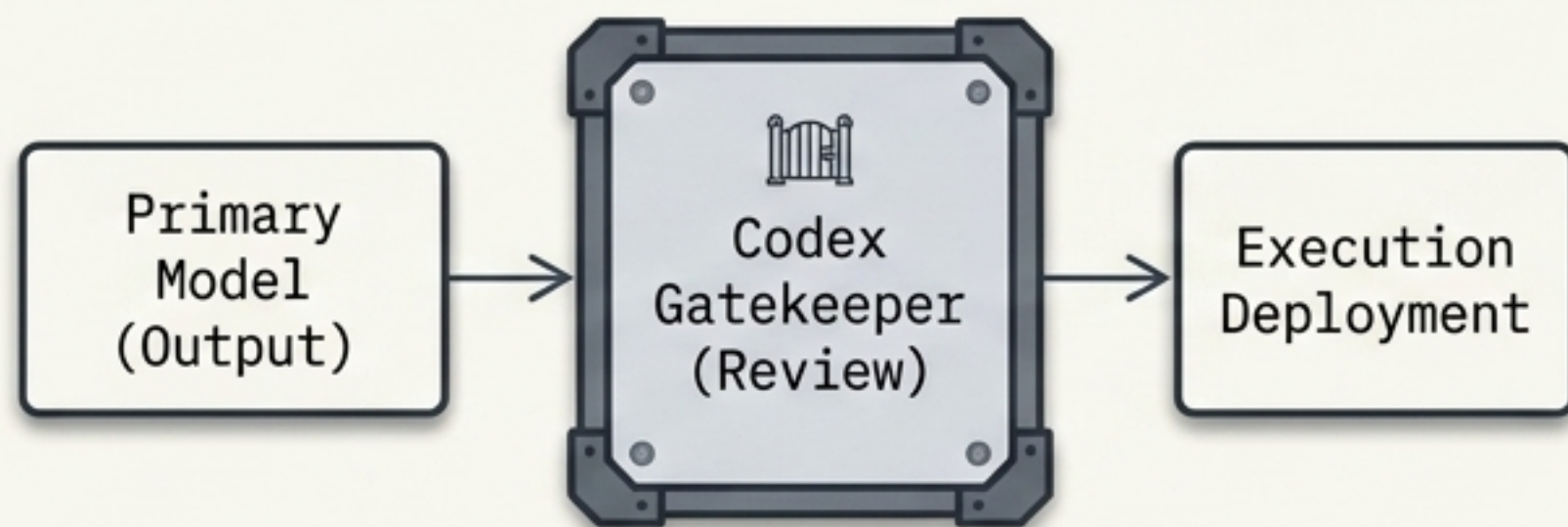
Blocks agents lacking a narrow scope and a testable hypothesis. Blocks ending a turn with unclosed checks.



Rule Engine: 73 rules with defined priorities. Any state-dependent rule without a hook is caught by an audit as an “orphan”.

Cross-Model Control & Observation

The Gatekeeper Protocol



Protocol: Codex/GPT reviews code changes before deployment to catch bugs the primary model missed. Managed by **5 auto-hooks**: post-edit reviews, prompt injections, prompt injections, blockers on unread reviews, verdict tracking, and an auto-call for help after 3 stalls.

June Volume Metrics

324

Total Reviews

1336

(689 Valid, 366 Stylistic, 278 False)

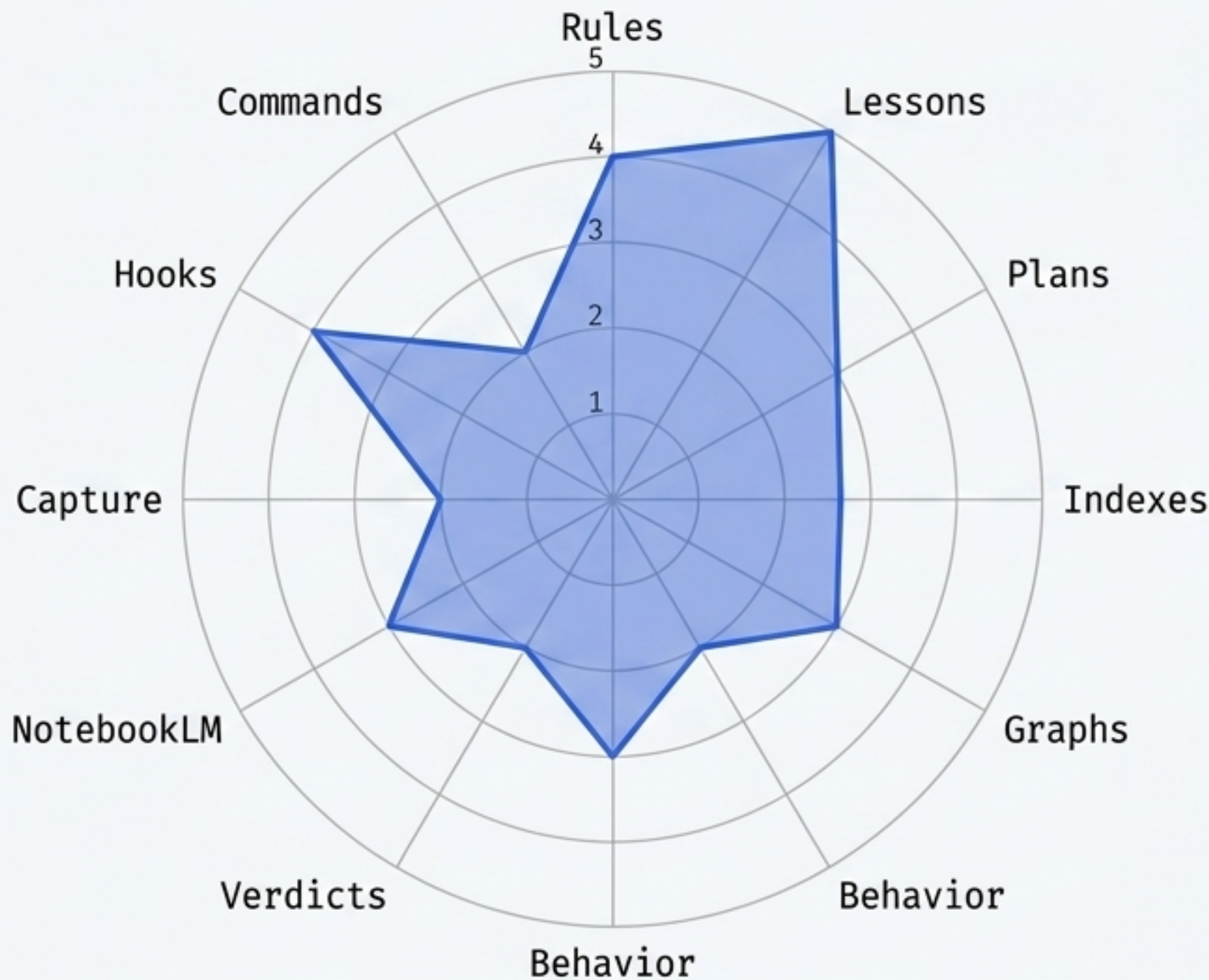
21%

Live false-positive rate
(measured by fact, not
hardcoded)

Financial Efficiency: Entire assistant overhead cost under \$2.00/month. Codex calls at ~\$0.05 per call.

Maturity by Domain: CMMI Assessment

System-wide CMMI Scale Average: 2.9 / 5.0



Level 4 – Managed, Measured

Rules and Lessons (Strictly enforced by machine via hard blockers).

Level 3 – Defined

Hooks, Commands, Dashboards, Plans, Indexes, Graphs.

Level 2 – Repeatable

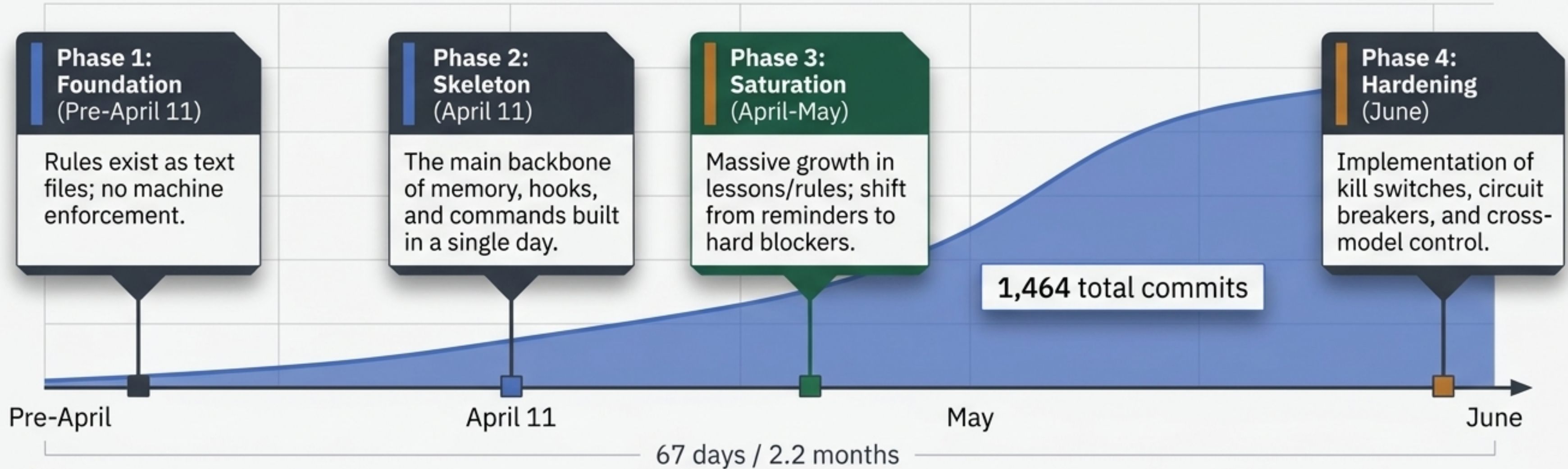
Capture, Verdicts, NotebookLM (Where the code-behavior gap persists).

Rule Coverage Synthesis

Out of 73 total rules, 22 are eligible for hard blockers (12 currently covered). The remaining 51 require qualitative judgment (style/assessment) and are routed to a judge model.

Core Insight: The primary limiter of the system is the gap where “code works, behavior lags”—treated strictly by turning reminders into hard blockers.

The 4-Phase Evolution & Production Reality



Production Context

Built alongside a portfolio of **14 applied projects** (3 complex, 4 medium, 7 simple). Complex flagships are live in production, verified by live network pings, valid certs, and automated deployment pipelines.

Post-Mortem: Honest Limitations & The Bottom Line

Honest Limitations



- **Bus Factor of 1:** Architecture depends entirely on one person and one Mac.



- **Over-Engineering Risk:** 357 hooks require maintenance; system counts firings but not utility per hook.



- **Meta-Work Conflict:** A significant portion of the 2.2 months went to tooling rather than product.



- **Self-Referential Metrics:** No external A/B comparison baseline.



- **System Drift:** Even self-documenting code drifts; documentation gaps were found during audits.

The Bottom Line



- In **67 days**, a fully autonomous, accumulative layer was built over Claude Code.



- Manages **74,800 lines of code** and **2,242 memory files** across 14 active projects.



- Successfully **shifts** AI operations from manual, amnesic prompting to an **automated** engine where 'memory is code.'



- The primary goal continues: relentlessly closing the '**behavior lags**' gap via machine enforcement.